

OpenTelemetry - Cheat Sheet

OpenTelemetry (OTel) is the vendor-neutral standard for telemetry data: traces, metrics, and logs. Born in 2019 from the merger of OpenTracing and OpenCensus. Hosted by the CNCF. Backed by every major observability vendor and cloud provider.

You instrument your code against the OTel API. The implementation (SDK + exporters) decides where the data goes: Jaeger, Tempo, Prometheus, Datadog, Honeycomb, your own Collector. Switching backends doesn't touch your code.

The pieces

Component	Role
API	Language-specific interface you call from app code (<code>Tracer</code> , <code>Meter</code> , <code>Logger</code>)
SDK	The implementation. Handles sampling, batching, exporting.
Instrumentation libraries	Pre-built spans for common libraries (HTTP, JDBC, Kafka, gRPC, Spring)
Exporters	Ship data to a backend (OTLP, Jaeger, Zipkin, Prometheus, vendor-specific)
Collector	Standalone process that receives, processes, and exports telemetry
Semantic conventions	Standardized attribute names (<code>http.method</code> , <code>db.system</code> , etc.)

The split between API and SDK is what makes vendor neutrality work. App code talks to the API. Configuration picks the SDK. The same code runs against any backend.

The three signals

OTel covers three signal types. Each has its own data model and lifecycle.

Traces

A trace is the lifecycle of a request across services. A span is one unit of work within a trace.

```
Trace abc123
  Span GET /orders/42 [0ms - 230ms]
    Span db.query users.findById [5ms - 15ms]
      Span http.call payment-service [20ms - 200ms]
        Span (in payment-service)
          Span db.query payments.insert [180ms - 190ms]
```

Each span has: name, start/end timestamps, attributes (key-value), status, links to other spans, events, parent.

Metrics

Time-series data. Same metric types as Prometheus, plus a few more.

- **Counter**: monotonic, increases only.
- **UpDownCounter**: goes up and down.
- **Histogram**: distribution of values.
- **Gauge**: observed value at a point in time.
- **ObservableGauge / ObservableCounter / ObservableUpDownCounter**: pulled via callbacks instead of pushed.

Logs

Structured log records with attributes. Tied to traces via the active context.

OTel logs are still maturing across languages. In Java, log records flow through SLF4J appenders or direct OTel API calls.

Context propagation

For a trace to span services, the trace context must travel with every call. OTel uses W3C Trace Context by default.

```
traceparent: 00-8f3a2bc4...-789xyz-01
tracestate:  vendor=value
baggage:     userId=42,region=us
```

- `traceparent` : version, trace ID, span ID, sampling flag.
- `tracestate` : vendor-specific extensions.
- `baggage` : arbitrary key-value pairs that propagate with the trace.

OTel's auto-instrumentation handles propagation for HTTP, gRPC, Kafka, messaging libraries. Manual propagation is needed for custom transports.

```
// Inject context into outbound headers
Map<String, String> headers = new HashMap<>();
GlobalOpenTelemetry.getPropagators()
    .getTextMapPropagator()
    .inject(Context.current(), headers, (h, k, v) -> h.put(k, v));

// Extract context from inbound headers
Context extracted = GlobalOpenTelemetry.getPropagators()
    .getTextMapPropagator()
    .extract(Context.current(), headers, new TextMapGetter<>() {
        public Iterable<String> keys(Map<String, String> h) { return h.keySet(); }
        public String get(Map<String, String> h, String k) { return h.get(k); }
    });
```

Resources

Every signal carries resource attributes: who's emitting this data?

```
service.name = "orders"
service.version = "1.4.2"
service.namespace = "shop"
deployment.environment = "prod"
host.name = "node-12"
k8s.pod.name = "orders-7b9c-x42a"
container.id = "abc123..."
```

OTel ships **semantic conventions** that standardize attribute names across languages and tools. Use them. A backend like Tempo, Datadog, or Honeycomb knows what `http.method` and `db.system` mean and builds UI around them.

```
Resource resource = Resource.getDefault()
    .merge(Resource.create(Attributes.builder()
        .put(ServiceAttributes.SERVICE_NAME, "orders")
        .put(ServiceAttributes.SERVICE_VERSION, "1.4.2")
        .put(DeploymentAttributes.DEPLOYMENT_ENVIRONMENT, "prod")
        .build()));
```

Sampling

Sending every trace is expensive. Sample.

Type	Behavior
AlwaysOn	Sample every trace. Dev or low-volume only.
AlwaysOff	Sample nothing.
TraceIdRatio	Random N% based on trace ID. Head-based, decided at trace start.
ParentBased	Inherit sampling decision from incoming <code>traceparent</code> . Standard for distributed propagation.
Tail-based (in the Collector)	Wait for the full trace, then decide based on errors, latency, attributes.

Production setups usually combine: head-based **ParentBased** + **TraceIdRatio** at the app, optional tail-based in the Collector for “always keep errors and slow traces”.

Exporters and OTLP

OTLP (OpenTelemetry Protocol) is OTEL’s native wire format. Binary protobuf over gRPC or HTTP. The default for most backends.

```
OTEL_EXPORTER_OTLP_ENDPOINT=https://otel-collector:4317
OTEL_EXPORTER_OTLP_PROTOCOL=grpc
```

Vendor exporters (Datadog, New Relic, Honeycomb) typically accept OTLP directly. Older backends (Jaeger UDP, Zipkin JSON) need their own exporter.

OpenTelemetry Collector

A standalone process that sits between your services and your backends. Receives, processes, and exports telemetry.

```
[your services] --OTLP--> [Collector] --OTLP--> [Tempo / Jaeger / Datadog / ...]
                        |
                        +--> [Prometheus] (for metrics)
                        +--> [Loki] (for logs)
```

Why use a Collector

- Decouples your apps from backend choice. Switch backends without redeploying services.

- Adds tail-based sampling, batching, retries.
- Strips PII or sensitive attributes before export.
- Fan-out to multiple backends (one trace to Jaeger and Datadog).
- Translates between formats (OTLP in, Prometheus out).

Anatomy

```
receivers:
  otlp:
    protocols:
      grpc:
        endpoint: 0.0.0.0:4317
      http:
        endpoint: 0.0.0.0:4318

processors:
  batch:
    timeout: 5s
  memory_limiter:
    check_interval: 1s
    limit_mib: 512
  tail_sampling:
    policies:
      - name: errors
        type: status_code
        status_code: { status_codes: [ERROR] }
      - name: slow
        type: latency
        latency: { threshold_ms: 1000 }
      - name: sample
        type: probabilistic
        probabilistic: { sampling_percentage: 5 }

exporters:
  otlp/tempo:
    endpoint: tempo:4317
  prometheusremotewrite:
    endpoint: https://prom.example.com/api/v1/write

service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [memory_limiter, tail_sampling, batch]
      exporters: [otlp/tempo]
    metrics:
      receivers: [otlp]
      processors: [memory_limiter, batch]
      exporters: [prometheusremotewrite]
```

Three pipeline stages:

- **Receivers** ingest data (OTLP, Prometheus scrape, Jaeger, Zipkin, syslog, host metrics).
 - **Processors** transform (batch, sample, redact, attribute manipulation, k8s metadata enrichment).
 - **Exporters** ship data out (OTLP, vendor-specific, file).
-

Java instrumentation

Auto-instrumentation

The OTel Java agent attaches at JVM start and instruments common libraries.

```
java -javaagent:opentelemetry-javaagent.jar \  
  -Dotel.service.name=orders \  
  -Dotel.exporter.otlp.endpoint=http://collector:4317 \  
  -Dotel.resource.attributes=deployment.environment=prod \  
  -jar orders.jar
```

Out of the box, it instruments Spring Boot, JDBC, HTTP clients (Apache, OkHttp, JDK HttpClient), Kafka, gRPC, JMS, MongoDB, Redis (Lettuce, Jedis), Cassandra, Elasticsearch, AWS SDK, and more.

Zero code changes for most apps. Traces appear automatically.

Manual instrumentation

Add the OTel API and SDK:

```
<dependency>  
  <groupId>io.opentelemetry</groupId>  
  <artifactId>opentelemetry-api</artifactId>  
</dependency>  
<dependency>  
  <groupId>io.opentelemetry</groupId>  
  <artifactId>opentelemetry-sdk</artifactId>  
</dependency>  
<dependency>  
  <groupId>io.opentelemetry</groupId>  
  <artifactId>opentelemetry-exporter-otlp</artifactId>  
</dependency>
```

Get a tracer and add custom spans:

```

Tracer tracer = GlobalOpenTelemetry.getTracer("orders");

Span span = tracer.spanBuilder("place-order")
    .setSpanKind(SpanKind.SERVER)
    .setAttribute("customer.id", customerId)
    .startSpan();

try (Scope scope = span.makeCurrent()) {
    // Any spans created from this thread become children of this span.
    Order order = service.place(req);
    span.setAttribute("order.id", order.getId());
} catch (Exception e) {
    span.recordException(e);
    span.setStatus(StatusCode.ERROR, e.getMessage());
    throw e;
} finally {
    span.end();
}

```

Annotations

The instrumentation annotations package adds `@WithSpan` :

```

import io.opentelemetry.instrumentation.annotations.WithSpan;
import io.opentelemetry.instrumentation.annotations.SpanAttribute;

@WithSpan("validate-order")
public boolean validate(@SpanAttribute("order.id") long orderId) {
    // ...
}

```

`@WithSpan` works when the Java agent is attached or when you wire in the annotations processor.

Metrics

```
Meter meter = GlobalOpenTelemetry.getMeter("orders");

LongCounter ordersPlaced = meter.counterBuilder("orders.placed")
    .setDescription("Orders placed")
    .setUnit("1")
    .build();

DoubleHistogram orderLatency = meter.histogramBuilder("orders.latency")
    .setDescription("Order placement latency")
    .setUnit("ms")
    .build();

// Record
ordersPlaced.add(1, Attributes.of(AttributeKey.stringKey("status"), "success"));
orderLatency.record(durationMs, Attributes.of(AttributeKey.stringKey("status"),
    "success"));
```

In Spring Boot, the Micrometer-to-OTel bridge translates Micrometer meters into OTel metrics. Less typing.

Semantic conventions

Names and meanings of attributes across languages and tools. A few important ones:

Convention	Attribute
HTTP server	<code>http.request.method</code> , <code>http.response.status_code</code> , <code>url.path</code> , <code>server.address</code>
HTTP client	<code>http.request.method</code> , <code>server.address</code> , <code>server.port</code>
Database	<code>db.system</code> , <code>db.operation</code> , <code>db.statement</code> , <code>db.name</code>
Messaging	<code>messaging.system</code> , <code>messaging.destination.name</code> , <code>messaging.operation</code>
RPC	<code>rpc.system</code> , <code>rpc.service</code> , <code>rpc.method</code>
Service	<code>service.name</code> , <code>service.version</code> , <code>service.namespace</code>
Deployment	<code>deployment.environment</code>
Host	<code>host.name</code> , <code>host.arch</code> , <code>os.type</code>
Kubernetes	<code>k8s.pod.name</code> , <code>k8s.namespace.name</code> , <code>k8s.node.name</code>
Container	<code>container.id</code> , <code>container.image.name</code>
Cloud	<code>cloud.provider</code> , <code>cloud.region</code> , <code>cloud.availability_zone</code>

Tools depend on these names. A trace with `http.method=GET` shows up correctly in any OTel-compatible UI. A trace with `myapp.http_method=GET` doesn't.

Stay current. The conventions evolved through 2024 (`http.method` was renamed to `http.request.method` , for example). The Java agent uses the new names by default.

OTel vs the alternatives

Aspect	OpenTelemetry	Datadog tracer	Jaeger client	Prometheus client
Coverage	Traces, metrics, logs	Traces, metrics, logs	Traces only	Metrics only
Vendor neutral	Yes	No	Yes	Yes (within Prom ecosystem)
Auto-instrumentation	Yes (Java agent)	Yes	Limited	No
Default backend	None (you pick)	Datadog	Jaeger	Prometheus
Stability (Java)	Stable for traces and metrics. Logs maturing.	Stable	Stable	Stable

The trend: most backends accept OTLP natively now. OTel has won the standard-protocol race. Vendor-specific agents still exist for richer features, but greenfield projects default to OTel.

Best practices

- **Use the Java agent for auto-instrumentation.** Free traces with no code changes.
- **Set `service.name` and `service.version` via resource attributes.** Without them, the trace explorer can't group anything cleanly.
- **Follow semantic conventions.** Custom attribute names break UI features in OTel-compatible backends.
- **Use ParentBased sampler in production.** Decisions made at the entry point propagate down the trace.
- **Add tail-based sampling at the Collector** for “always keep errors and slow traces” without paying for everything.
- **Run a Collector.** Even when shipping to a SaaS backend, the Collector buys you decoupling and per-tenant control.
- **Span at meaningful boundaries.** Request handler, DB call, external HTTP, business operations. Spans cost CPU and memory.
- **Set `SpanKind` correctly.** SERVER for inbound, CLIENT for outbound, PRODUCER and CONSUMER for messaging, INTERNAL for in-process.
- **Record exceptions with `recordException` and set `StatusCode.ERROR`.** Backends count error spans for alerts.

- **Use baggage sparingly.** Every baggage entry rides with every downstream call. PII in baggage leaks across services.
-

Common gotchas

- **`Span.end()` not called.** The span never gets exported. Wrap span work in try-finally.
 - **Wrong context propagation across async boundaries.** Submitting work to an `ExecutorService` doesn't carry the current span by default. Use `Context.taskWrapping(executor)` or `Context.current().wrap(runnable)`.
 - **Auto-instrumentation collides with manual.** Adding a custom `@WithSpan` inside an auto-instrumented method gives two spans for the same work. Audit the trace.
 - **Sampler set per-service.** A high upstream sample rate plus a low downstream rate breaks traces in the middle. Use `ParentBased` to keep it consistent.
 - **Resource attributes too narrow.** Forgetting `deployment.environment` makes prod and staging look the same.
 - **Custom attribute names that conflict with semantic conventions.** Future tooling may overwrite or ignore them.
 - **Tail-based sampling memory.** The Collector holds spans in memory until the trace is “complete enough” to decide. Misconfigured, this OOMs.
 - **OTel Java agent class loader issues.** The agent runs ahead of your classloader. Conflicts with security managers or non-standard classloaders show up as obscure stack traces.
 - **Logs without trace correlation.** Without the log instrumentation, log lines won't have `trace_id` / `span_id`. Configure your logger to inject them.
 - **Cardinality blowup on metrics.** Same Prometheus warning. OTel doesn't save you from putting `user_id` as an attribute.
 - **`@WithSpan` without the agent.** Without the Java agent, the annotation does nothing. Verify by checking that spans actually appear.
-

Quick reference

Concept	Key fact
Signals	Traces, metrics, logs
API vs SDK	Code against API, configure SDK
Span	One unit of work within a trace
Trace	The lifecycle of a request across services
Resource	Attributes describing the producer (service, host, env)
W3C <code>traceparent</code>	Standard header for context propagation
OTLP	OTel's native wire protocol (gRPC or HTTP)
Collector	Receives, processes, exports telemetry
Receiver / Processor / Exporter	Collector pipeline stages
ParentBased sampler	Inherits decision from <code>traceparent</code>
Tail-based sampling	Decide after the trace completes (in the Collector)
Semantic conventions	Standardized attribute names
Java agent	<code>-javaagent:opentelemetry-javaagent.jar</code> for auto-instrumentation
<code>@WithSpan</code>	Annotation for custom spans (requires agent or processor)
<code>Span.recordException</code>	Mark an exception on the active span
Baggage	Key-value pairs that propagate with the trace context